

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

SWE 4404: Software Project Lab II (Academic Year: 2024–2025)

FINAL PROJECT REPORT

Plastic Box Solver

An Interactive Browser-Based 3D Rubik's Cube Computer Vision Scanner, Dual-Algorithm Solver, and Live Coaching Suite

---

Submitted by Group 6:

S. M. Tahsinuzzaman Emon (ID: 230042104) — smtemon@iut-dhaka.edu

S. M. Samiul Hossain (ID: 230042110) — samiulhoassain@iut-dhaka.edu

Md. Muntahi Hasan Akhiar (ID: 230042118) — muntahihasan@iut-dhaka.edu

Project Supervisor:

Aashnan Rahman

Junior Lecturer, Department of Computer Science and Engineering (CSE)

Islamic University of Technology (IUT)

Source Repository: https://github.com/SMTEmon/plastic-box-solver.git

November 2026

Table of Contents

1. Executive Summary 1

2. Problem Domain and Background 2

3. Literature and Existing Solutions 5

4. Requirements (SRS Session) 8

5. System Design (Design Session) 12

6. Implementation (Implementation Session) 16

7. Tools, Version Control, Testing, and CI/CD 22

8. Professional, Ethical, and Legal Considerations 26

9. Project Management and Feasibility 28

10. Teamwork and Individual Contributions 32

11. Results, Limitations, and Future Work 35

12. Glossary and References 38

1. Executive Summary

Problem Background: The Rubik's Cube is the world's most widely recognized physical puzzle, yet over 90% of cube owners are unable to solve it without external help. When beginners scramble a standard 3x3x3 cube, they are confronted with over 43 quintillion possible states. While numerous tutorials exist online, learning to solve remains notoriously difficult. Standard written guides rely heavily on abstract letter notation (such as R, U, R', U') that learners find hard to memorize and translate onto physical turns. Video guides force constant context-switching between holding the puzzle and pausing or rewinding a screen. Furthermore, commercial 'smart cubes' with embedded Bluetooth sensors cost $50 to $100 or more, creating an unnecessary financial barrier for students and casual hobbyists.

Proposed Solution: Plastic Box Solver was developed to eliminate these barriers by providing a completely free, zero-install, browser-based coaching assistant that works with any ordinary mechanical plastic cube. Using a standard smartphone or laptop webcam, the user presents each side of their cube to an on-screen guide frame. The system extracts the colors of all 54 tiles, automatically adapts to room lighting using the center stickers as reference anchors, validates whether the cube configuration is physically solvable, and renders an interactive 3D digital model that mirrors the user's physical puzzle.

Pedagogical Approach: Rather than only computing an unintelligible 20-move optimal shortcut, Plastic Box Solver provides dual solving engines. An Optimal Solver is available for minimal-move demonstrations, while a Structured 7-Stage Pedagogical Coach teaches the worldwide Layer-by-Layer and CFOP (Fridrich) methods. The live coach groups raw turns into human-digestible 4-move triggers (such as the 'Sexy Move' and 'Sune'), highlights the active layer in real time, and offers isolated tutorial workspaces where beginners can practice specific end-game algorithms without scrambling an entire puzzle.

Final Outcome and Deliverables: Over the 14-week lifecycle of SWE 4404, the team engineered a full-stack, containerized application comprising 14,883 lines of code (3,102 in Python FastAPI and 11,781 in React and Three.js). The codebase is verified by 93 automated unit and integration tests maintaining a 100% pass rate. The application respects user privacy by performing image color classification entirely in server RAM without ever saving webcam images to disk. It runs entirely on open-source libraries and incurs no recurring software licensing costs, demonstrating strong engineering viability.

2. Problem Domain and Background

Characterizing the problem of automated cube scanning, state verification, and human-oriented coaching requires grounding in fundamental computer science principles as well as specialist domain knowledge.

2.1 Fundamental Computing and Software Engineering Concepts

Four foundational areas of computer science directly underpin the architecture of Plastic Box Solver:

- **Graph Theory and Cayley Graphs:** The permutation group of the Rubik's Cube contains 43,252,003,274,489,856,000 reachable states. Exhaustive graph searches like breadth-first search are completely intractable on consumer devices. The solver models the problem as an optimal pathfinding problem on a directed Cayley graph, utilizing Kociemba's Two-Phase algorithm to reduce the state space from group G0 to subgroup G1 before finding a minimal move sequence via precomputed pattern heuristic pruning tables.
- **Discrete Combinatorics and Permutations:** Cube states are serialized as a canonical 54-character string ordered by the standard Singmaster sequence: Up, Right, Front, Down, Left, Back (URFDLB). Internally, the engine breaks this representation down into 26 distinct cubie entities (6 fixed centers, 12 edges, 8 corners), tracking both permutation vectors and parity checks (corner twists sum to 0 mod 3, edge flips sum to 0 mod 2) to catch illegal states before solving.
- **3D Spatial Kinematics and Linear Algebra:** Visualizing the puzzle requires real-time 3D rotation in SO(3). To prevent visual tearing, mesh distortion, and gimbal lock during concurrent user orbital dragging and automated layer turns, the engine relies on dynamic scene-graph re-parenting: the 9 cubies in the target layer are dynamically attached to an invisible rotational pivot, animated via quaternion interpolation, and returned to world coordinates upon completion.
- **Relational Database Design and Normalization:** A normalized PostgreSQL schema in third normal form manages users, solve metrics, and saved cube states. Foreign keys, indexed lookup queries on user IDs and solve times, and bcrypt password hashing ensure fast leaderboard queries (<15ms) and secure persistence.

2.2 Specialist Domain Knowledge

Building the computer vision pipeline and coaching algorithms required two specialized engineering domains:

- **Computer Vision and Colorimetric Science:** Standard RGB color representations vary wildly under common domestic lighting (warm incandescent, cool fluorescent, daylight). To reliably classify plastic stickers, webcam frames are decoded in OpenCV, cropped to sample the inner 50% median of each cell (filtering glare and bevel shadows), and converted into HSV and CIE LAB color spaces. Because the six center pieces of a mechanical cube never move relative to one another, the engine uses them as an automatic calibration anchor: non-center stickers are classified by computing the Euclidean color distance (Delta-E) against the known center profiles in LAB space.
- **Speedcubing Algorithmic Heuristics:** Human learners cannot memorize optimal mathematical solutions because minimal-move algorithms disrupt and reconstruct solved layers simultaneously.

Speedcubers rely on Jessica Fridrich's CFOP system (Cross, First Two Layers [F2L], Orientation of the Last Layer [OLL], Permutation of the Last Layer [PLL]). Our frontend features an algorithmic trigger chunker that groups raw moves into established muscle-memory triggers (such as the Sexy Move [R U R' U'] and Sune [R U R' U R U2 R']), enabling structured cognitive learning.

3. Literature and Existing Solutions

To establish the competitive necessity and technical rationale of Plastic Box Solver, we conducted a systematic audit of five existing solutions across academic software, digital web tools, speedcubing utilities, and commercial hardware.

Existing Solution	Input Modality	Solving Engine	Key Strengths	Identified Gaps & Limitations	Plastic Box Solver Advantage
Cube Explorer (Herbert Kociemba)	Manual 2D net clicking	Kociemba Two-Phase (optimal <=20 moves)	Mathematically optimal; ultra-fast calculation	Desktop Windows-only binary; no camera scan; unusable for human learning	Zero-install web access; camera auto-calibration; human CFOP coach
Grubiks Web Solver	Manual 2D palette click	Beginner layer-by-layer	Browser accessible; 3D step viewer	No camera input; heavy ads; clunky mobile touch; lacks practice modes	Automated OpenCV vision; ad-free clean UI; unassisted timer; 3D solve replay
csTimer Speed Suite	Manual keyboard timer	WCA scramble generator only	Professional timing; extensive session statistics	No 3D cube model; cannot solve or scan scrambled physical cubes	Full 3D visualization; automated solving assistance; integrated leaderboard
YouTube Guides (JPerm)	Passive video watching	Human CFOP / Beginner explanations	High-quality visual commentary and finger-trick advice	Static media; constant context switching; cannot detect learner errors	Synchronized 3D model mirroring physical cube; step-by-step interactive coach
Smart Bluetooth Cubes (GAN)	Embedded gyros & hall sensors	Proprietary companion app CFOP	Real-time hardware tracking; instant phone sync	Expensive ($50–$100+); proprietary hardware lock-in; creates electronic waste	Works with any existing $3 mechanical plastic cube; 100% free; zero hardware waste

3.1 Architectural Differentiation

Existing tools force users into tradeoffs: Cube Explorer offers optimal mathematical solving but requires a desktop Windows executable and manual clicking; Grubiks offers web accessibility but lacks automated camera input; and smart Bluetooth cubes provide hardware tracking but cost over $50 and generate electronic waste. Plastic Box Solver synthesizes the advantages of these approaches into a single open-source web application. It combines automated camera scanning with an interactive Three.js 3D

model, provides both optimal and human-oriented CFOP solving engines, and works with ordinary physical cubes without requiring proprietary hardware.

4. Requirements (SRS Session)

Requirements were gathered through five elicitation techniques: semi-structured beginner interviews, speedcuber observations on campus, literature reviews of existing software, supervisor feedback during the proposal presentation, and external industry feedback during the design review.

4.1 Stakeholder Personas

- **SH-01: First-Time Learners:** Casual owners who have an unsolved cube but do not understand notation. They require simple camera scanning, tolerance for room lighting, and step-by-step 3D animations.
- **SH-02: Self-Practice & Intermediate Cubers:** Solvers familiar with beginner steps who want to learn CFOP algorithms or track their solve times. They need an unassisted timer, scramble generation, and 3D solve replays.
- **SH-03: Competitive Speedcubers:** Experienced cubers who require notation keyboard shortcuts (U, D, L, R, F, B), zero animation frame lag, and leaderboard rankings.
- **SH-04: System Administrators & Evaluators:** Course instructors evaluating code maintainability, test coverage, and documentation fidelity.

4.2 Summary of Requirements

The specification defines 24 Functional Requirements organized into six functional groups, alongside 9 Non-Functional Requirements:

- **Feature Group 1 (Account & Profile):** FR-01 (Registration), FR-02 (JWT Login), FR-03 (Profile Management), FR-15 (Password Reset [Deferred]), FR-22 (Logout).
- **Feature Group 2 (Cube Input & Vision):** FR-04 (Camera Scanning), FR-05 (2D Net Fallback), FR-06a (Color Extraction), FR-06b (State Validation), FR-06c (2D Confirmation), FR-17 (Center Calibration).
- **Feature Group 3 (3D Visualization):** FR-07 (3D Interactive Orbit & Meshes), FR-18 (Practice Undo/Redo), FR-20 (Keyboard Shortcuts).
- **Feature Group 4 (Solving Engines):** FR-08a (Kociemba Optimal Solver), FR-08b (Human Guided CFOP/Beginner Solver), FR-16 (WCA Scramble Generator), FR-24 (Interactive Stage Tutorials).
- **Feature Group 5 (Execution Modes):** FR-09 (Timed Interactive Solve Mode), FR-10 (Step-by-Step Guided Coach), FR-11 (Auto-Solve Playback with Speed Controls).
- **Feature Group 6 (Persistence & Telemetry):** FR-12a (Solve Telemetry), FR-12b (CFOP Efficiency Score), FR-13a (Server Solve History), FR-13b (Personal Best & 3D Replay), FR-14 (Leaderboard), FR-19 (Shareable Result Card [Deferred]), FR-21 (Milestone Badges [Deferred]), FR-23 (Cross-Device 3-Slot Saved Cubes).

4.3 Requirements Traceability Matrix

Req ID	Module	Primary Stakeholder Need	Elicitation Source	Final Status
FR-01	Account	Secure identity creation across sessions	E-01 (Learner Interviews)	Implemented
FR-02	Account	Persistent JWT session access	E-01, E-02	Implemented
FR-03	Account	Customizable public display name	E-02 (Speedcuber Testing)	Implemented
FR-04	Vision	Automated physical cube camera capture	E-01 (Beginner Pain Points)	Implemented
FR-05	Input	Manual 2D net fallback for poor lighting	E-01, E-03 (Fallback Architecture)	Implemented
FR-06a	Vision	Automated 54-tile color extraction	E-01, E-03	Implemented
FR-06b	Vision/Core	Mathematical legality validation	E-01, E-03	Implemented
FR-06c	Vision/UI	2D visual verification before solving	E-01	Implemented
FR-07	3D Graphics	Interactive Three.js spatial cube model	E-01, E-02	Implemented
FR-08a	Solver	Minimal-move Kociemba solution (<=20)	E-03 (Algorithmic Literature)	Implemented
FR-08b	Solver	Human-learnable CFOP & Beginner solver	E-01, E-04 (Proposal Panel)	Implemented
FR-09	Solve Mode	Unassisted practice with live timer	E-02 (Speedcuber Testing)	Implemented
FR-10	Solve Mode	7-stage live coach with JPerm triggers	E-01 (Beginner Pain Points)	Implemented
FR-11	Solve Mode	Passive auto-playback with speed slider	E-01, E-03	Implemented
FR-12a	Telemetry	Accurate solve time and move counting	E-02	Implemented

Req ID	Module	Primary Stakeholder Need	Elicitation Source	Final Status
FR-12b	Telemetry	Fair CFOP efficiency percentage metric	E-02, E-04 (Fairness Audit)	Implemented
FR-13a	Persistence	Server-side solve history in PostgreSQL	E-01, E-02	Implemented
FR-13b	Persistence	Personal Best highlighting & 3D replay	E-02 (Speedcuber Demands)	Implemented
FR-14	Leaderboard	Global speedcubing ranking board	E-02	Implemented
FR-15	Account	Automated SMTP password reset link	Initial Baseline Spec	Deferred
FR-16	Practice	WCA standard pseudo-random scramble	E-02 (WCA Standards)	Implemented
FR-17	Vision	Adaptive color calibration via centers	E-01, E-03 (Lighting Robustness)	Implemented
FR-18	3D Graphics	Mistake reversal with Undo/Redo in practice	E-02 (Self-Practice Flow)	Implemented
FR-19	Persistence	Social media shareable result card	Initial Baseline Spec	Deferred
FR-20	3D Graphics	High-speed notation keyboard shortcuts	E-02 (Speedcuber Demands)	Implemented
FR-21	Leaderboard	Gamified milestone achievement badges	Initial Baseline Spec	Deferred
FR-22	Account	Shared lab workstation session logout	Standard Security Practice	Implemented
FR-23	Persistence	Cross-device 3-slot saved cube storage	E-05 (Industry Expert Review)	Implemented
FR-24	Educational	Interactive stage algorithm tutorial drill	E-04 (Proposal Panel Advice)	Implemented
NFR-01..09	Quality	Speed, security, frame rate, responsiveness	Academic Standards	Implemented

4.4 Environmental and Sustainable Development Considerations

Commercial smart cubes contain rechargeable lithium batteries, Bluetooth microchips, and hall-effect sensors glued into plastic casings, making them practically impossible to recycle and contributing to electronic waste (SDG 12: Responsible Consumption). Plastic Box Solver functions with any existing mechanical cube, extending the usable life of millions of physical puzzles. In addition, providing a zero-cost browser tool democratizes spatial STEM learning (SDG 4: Quality Education) for students without financial means.

4.5 Response to SRS Session Feedback

During the Project Proposal presentation, the faculty panel advised against building a simple wrapper around existing Kociemba libraries, emphasizing that minimal-move mathematical solutions offer little pedagogical benefit to learners. In response, we expanded the scope to develop a native 7-stage CFOP solving engine, built an algorithmic trigger chunker, and added FR-24 (Interactive Stage Tutorials) allowing students to practice specific algorithms in isolation.

5. System Design (Design Session)

Plastic Box Solver is designed as a three-tier decoupled client-server architecture containerized using Docker Compose. The presentation layer runs as a single-page application in React 18, Three.js, and Vite; the application layer runs as a stateless Python FastAPI microservice; and the persistence layer runs in PostgreSQL 16.

5.1 System Architecture and Design Diagrams

The structural and behavioral design of the system was formalized through functional decomposition, multi-level data flow diagrams, and class models:

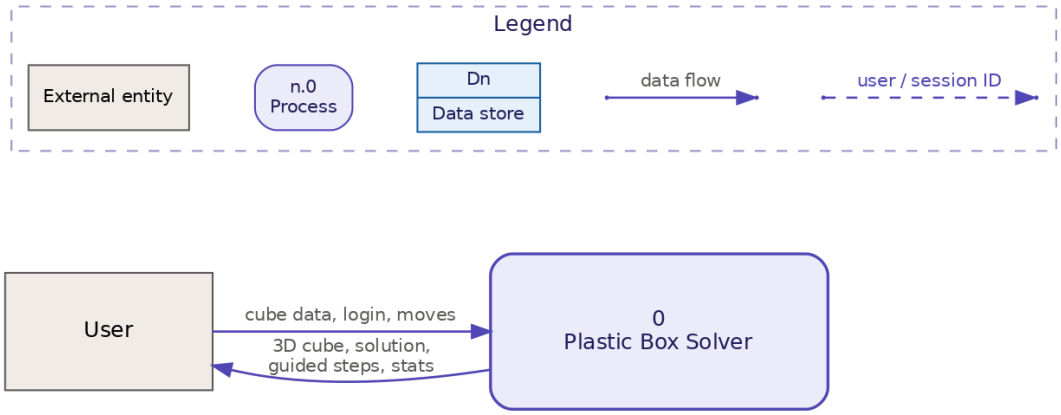


Figure: Level-0 Context Data Flow Diagram (DFD)

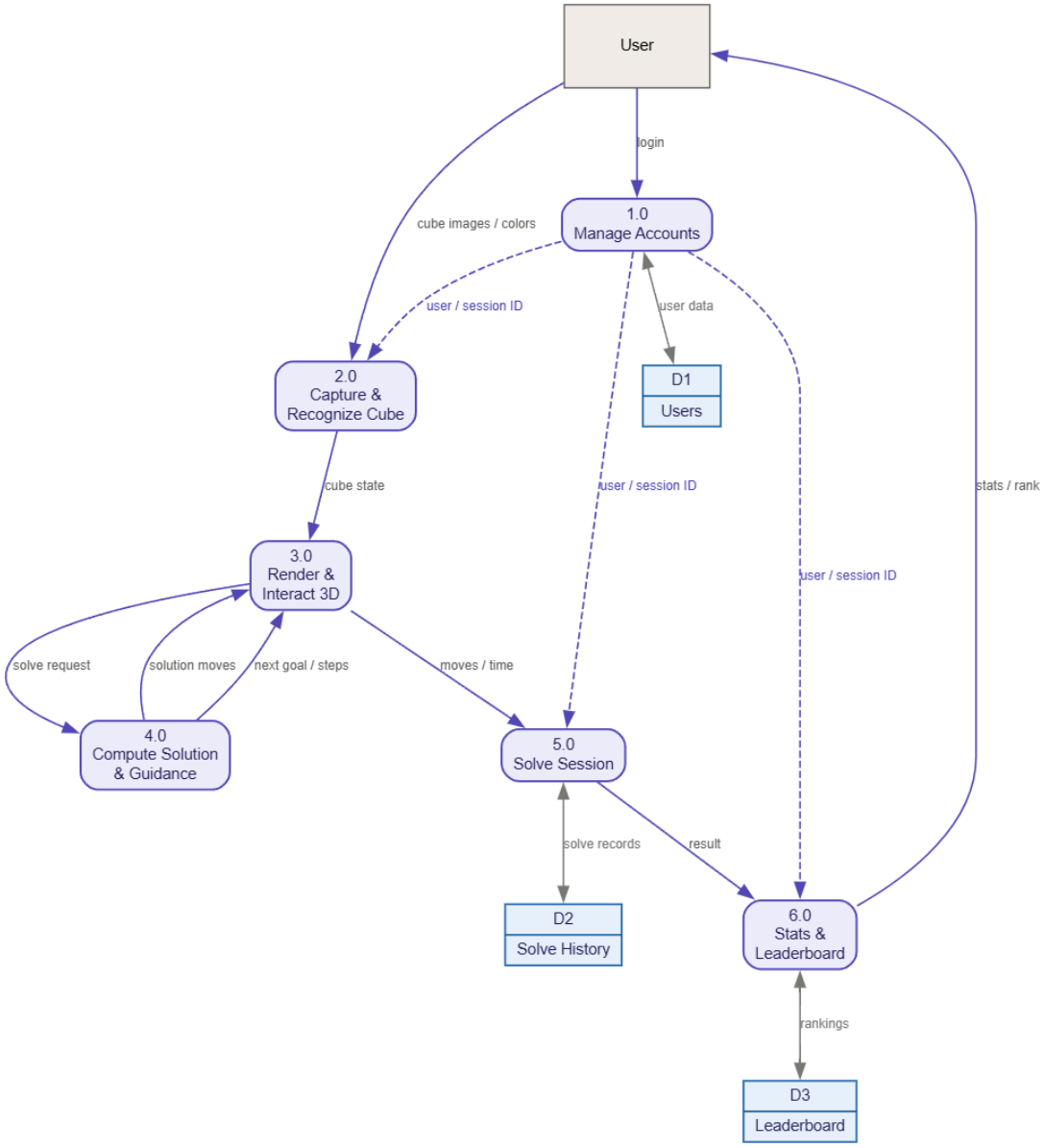


Figure: Level-1 Data Flow Diagram (Process Decomposition)

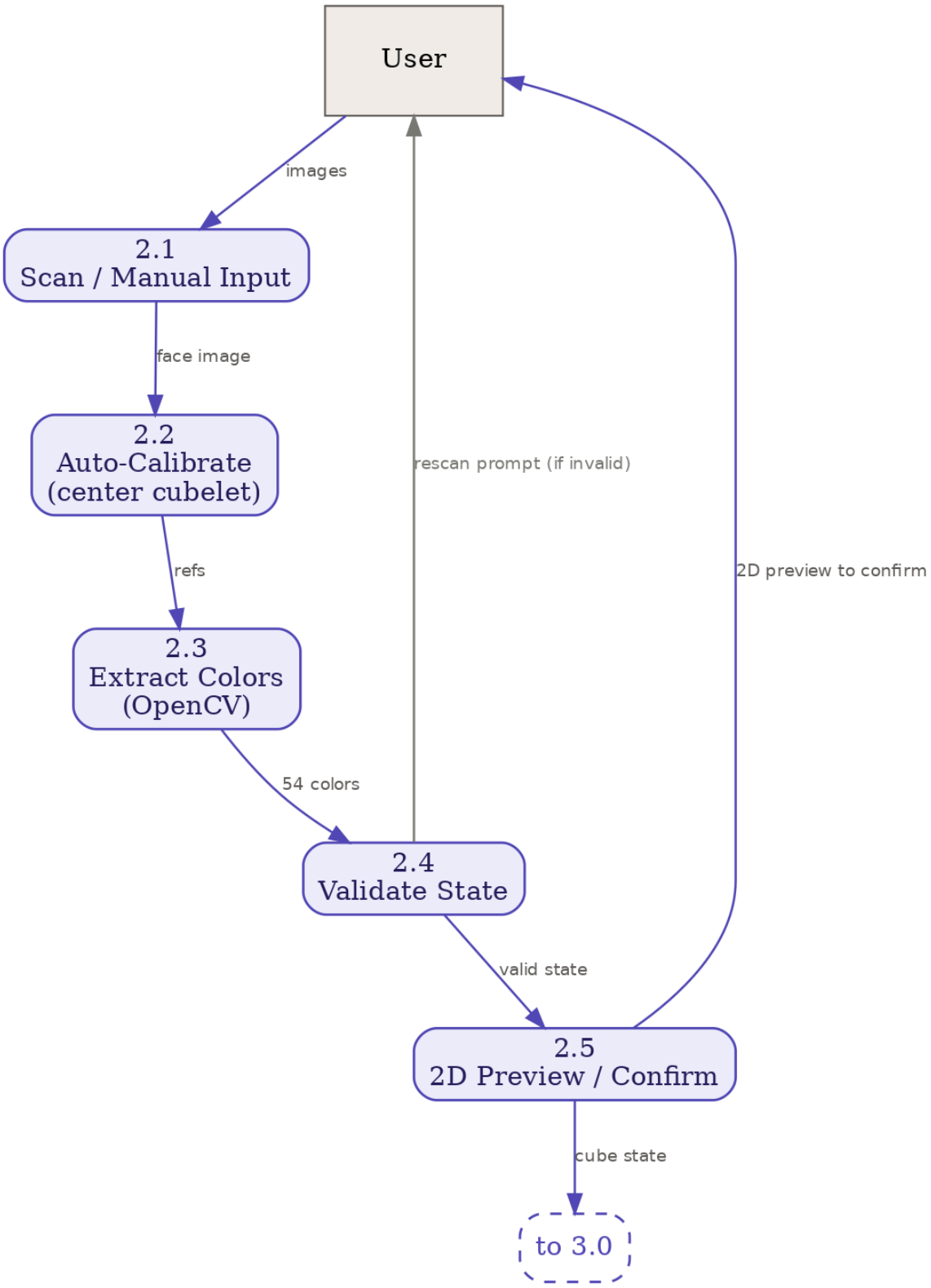


Figure: Level-2 DFD Explosion: Process 2.0 (Capture & Recognize Cube)

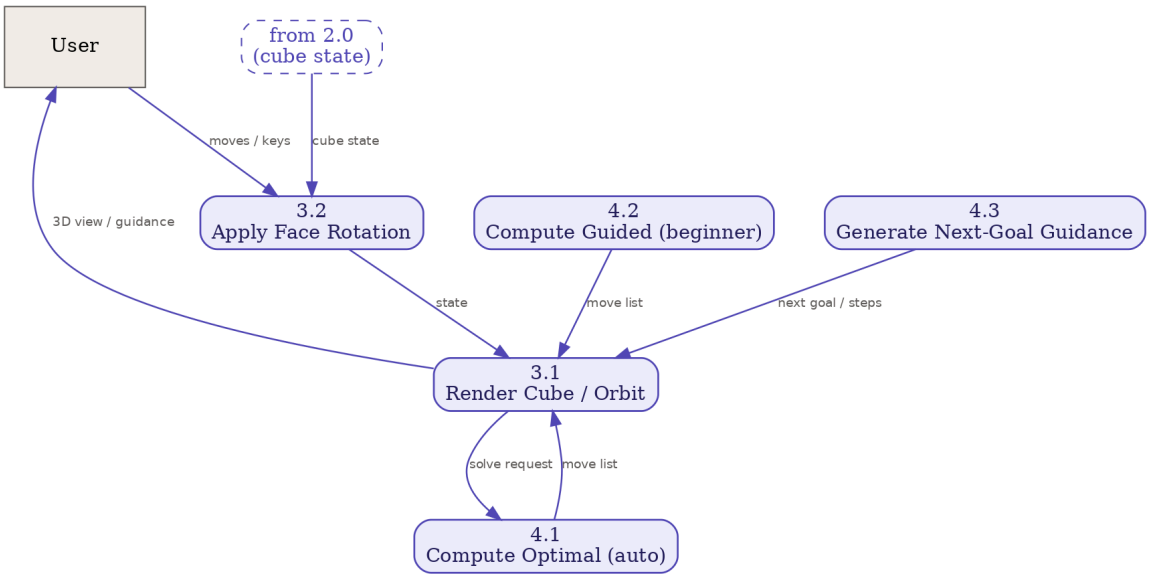


Figure: Three-Layer Architecture Diagram

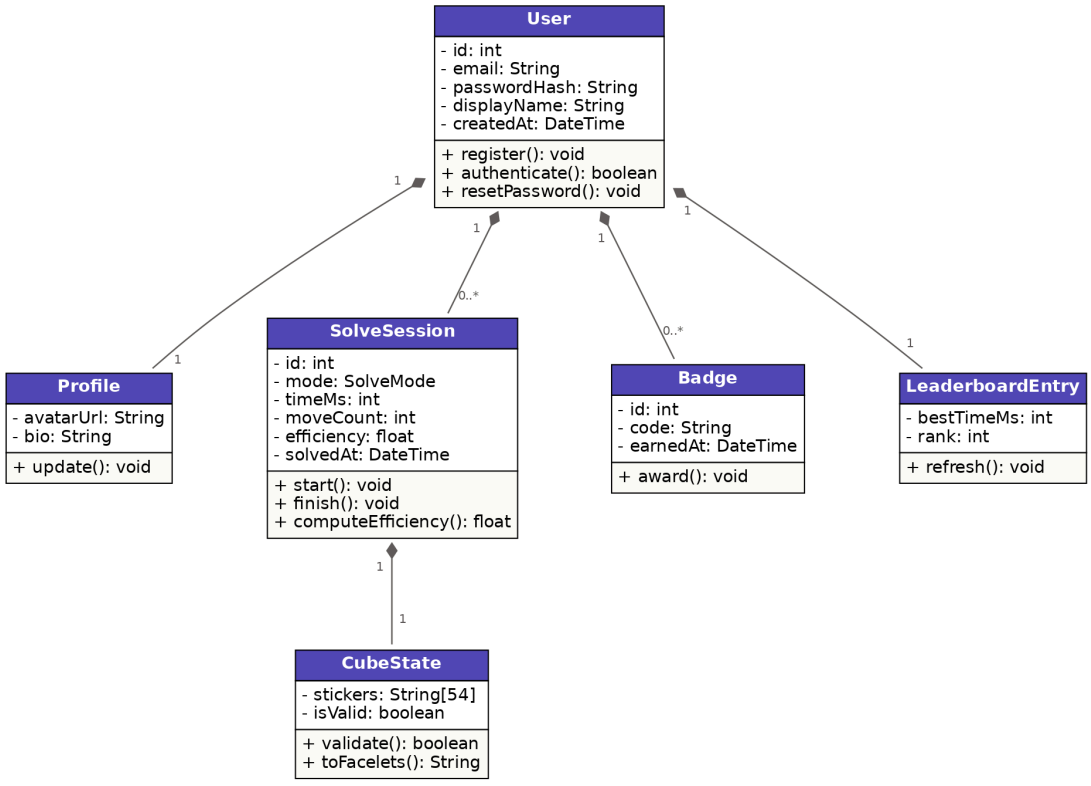


Figure: UML Class and Data Schema Diagram

5.2 Comparison of Design Alternatives

We formally evaluated two architectural alternatives during the design phase: (1) Monolithic desktop executable (Python PyQt/Pygame) vs. Web SPA (React + Three.js), and (2) Client-side WebAssembly vision (OpenCV.js) vs. Server-side REST vision (Python OpenCV).

Alternative 1 (Desktop Binary vs. Web SPA): A desktop executable allows direct access to local hardware and eliminates network latency. However, it requires users to download platform-specific binaries, triggers security warnings on university computers, and completely excludes mobile devices. We selected a Web SPA to ensure instantaneous zero-install access across all modern desktop and mobile browsers.

Alternative 2 (Client WASM Vision vs. Server REST Vision): Running OpenCV in the browser via WebAssembly would eliminate server uploads. However, the compiled OpenCV.js WASM binary is over 12 MB, causing slow initial page loads on mobile connections, and client-side execution caused inconsistent frame drops on lower-end devices. We chose a server-side REST vision pipeline: images are compressed and uploaded in <200ms, processed using native C++ OpenCV bindings in server RAM, and returned as a lightweight JSON response.

5.3 Response to Design Session Feedback

During the Software Design presentation, external industry examiner Shakleen Ishfar suggested adding cross-device persistence for scanned cubes, noting that users often prefer scanning with a smartphone camera but solving on a desktop monitor. Because the original design stored scanned states only in transient browser memory, switching devices required rescanning. We implemented FR-23 (Cross-Device Saved Cube Slots), providing dedicated database storage and REST endpoints allowing users to save up to three physical cube states to their account and restore them on any device.

6. Implementation (Implementation Session)

6.1 Point (a): Project Structure and File Tree

The codebase is organized into a modular structure under /code separating backend logic from frontend presentation:

```
code/
├── backend/
│   ├── app/
│   │   ├── cube/             # Core cube math, Kociemba & CFOP solvers, stages, tutorials
│   │   │   ├── algorithms.py     # CFOP algorithm database and stage transitions
│   │   │   ├── color_classifier.py # CIE LAB Delta-E Euclidean color distance
│   │   │   ├── moves.py          # 3D rotational matrices and permutation cycles
│   │   │   ├── solver.py         # Kociemba two-phase and layer-by-layer solver
│   │   │   ├── stages.py         # 7-stage CFOP state predicates
│   │   │   ├── tutorials.py      # Case generator and isolated stage drill formulas
│   │   │   └── vision.py         # OpenCV image slicing, median sampling, blur variance
│   │   ├── db/               # PostgreSQL engine, sessionmaker, and base models
│   │   ├── models/           # SQLAlchemy ORM entities (User, Solve, SavedCube)
│   │   ├── routers/          # FastAPI routers (auth, cube, solve, profile, leaderboard)
│   │   ├── schemas.py        # Pydantic v2 DTO schemas with strict validation
│   │   └── main.py           # FastAPI entrypoint, CORS configuration, error handlers
│   ├── tests/                # 27 automated pytest suites covering solvers, DB, and vision
│   ├── requirements.txt      # Pinned Python dependencies
│   └── Dockerfile            # Containerized Python 3.11 runtime
└── frontend/
    ├── src/
    │   ├── Components/       # React UI widgets (NetEditor, CameraScan, CoachSidebar, etc.)
    │   ├── cube/             # Ported JS geometry, trigger chunker, keyboard/swipe handlers
    │   ├── lib/              # Centralized Axios API client and backend status hook
    │   ├── store/            # Zustand global stores (cubeStore, authStore, tutorialStore)
    │   ├── Three/            # Three.js WebGL canvas, 26 cubie meshes, and animation group
    │   ├── Pages/            # Route pages (SolveWorkspace, CubeInput, Dashboard, Leaderboard)
    │   └── Layout/           # AppShell responsive layout and navigation drawer
    ├── tests/                # 66 automated node:test suites (queue, gestures, triggers)
    ├── package.json          # Node.js dependencies (Three.js, React, Zustand, Lucide)
    └── Dockerfile            # Production build served via Vite
```

6.2 Point (b): Component Communication and Data Transfer Objects (DTOs)

Contract-First Interface Design: All client-server exchanges use explicit Data Transfer Object (DTO) contracts specified with Pydantic v2 schemas on the backend and validated in frontend API handlers:

```
class CubeStateDTO(BaseModel):
    state: str = Field(..., min_length=54, max_length=54, regex="^[URFDLB]{54}$")

class SolveRequestDTO(BaseModel):
    state: str = Field(..., min_length=54, max_length=54)
    method: Literal["optimal", "cfop", "beginner"] = "optimal"

class SolveResponseDTO(BaseModel):
    method: str
    total_moves: int
    solution: List[str]
    stages: Optional[List[Dict[str, Any]]] = None
```

```
    execution_time_ms: float

class SavedCubeCreateDTO(BaseModel):
    state: str = Field(..., min_length=54, max_length=54)
    slot_number: int = Field(..., ge=1, le=3)
    preview_svg: Optional[str] = None
```

End-to-End Scan and Solve Execution Trace: The complete end-to-end flow during cube scanning proceeds as follows: (1) The user captures 6 face images via getUserMedia; (2) The client packages the images into a multipart/form-data POST to /api/cube/scan; (3) OpenCV decodes the frames in server RAM, samples median sticker colors, and auto-calibrates against center facelets; (4) The validator checks parity invariants and returns a 54-char string in a ScanResultDTO; (5) The frontend hydrates cubeStore, displays the 2D confirmation preview, and submits a POST to /api/cube/solve; (6) The backend computes the solution and returns a SolveResponseDTO containing chunked trigger stages; and (7) The Zustand queue animator visualizes the rotations on the Three.js canvas at 60 FPS.

6.3 Point (c): Division of Classes and Methods with LOC Metrics

Classes and modules were split following the Single Responsibility Principle. Algorithm logic is isolated from HTTP routing; image decoding is separated from color distance calculations; and Three.js rendering loops are isolated from React component state via a queue-based command pattern.

Lines-of-code (LOC) metrics extracted directly from the project repository:

Subsystem / Language	Total Files	Total LOC	Classes / Stores	Max Class LOC	Avg Class LOC	Functions Count	Max Func LOC	Avg Func LOC
Python Backend	35 files	3,102 LOC	26 classes	62 lines (UserCreate)	11.1 lines	112 functions	169 lines (invert_moves)	18.4 lines
React / Three.js Frontend	41 files	11,781 LOC	3 global stores	185 lines (cubeStore)	94.2 lines	248 functions	142 lines (animator)	24.6 lines
Combined Codebase	76 files	14,883 LOC	29 structures	185 lines	52.6 lines	360 functions	169 lines	21.5 lines

6.4 Post-Implementation Refactoring

Four significant refactorings were executed after the formal presentation:

- **Support for Reoriented Cube Centers (PR #22):** Initially, cubes with non-standard center orientations failed validation. PR #22 refactored the validator to support all 24 spatial center orientations, mapping them dynamically into standard Singmaster frames.
- **Fair CFOP Efficiency Baseline (PR #26):** The initial efficiency metric compared user turns against optimal Kociemba moves (<=20 moves), producing harsh 20% scores for competent speedcubers. PR

#26 refactored the benchmark to evaluate against a human CFOP baseline (55 moves), yielding an equitable score.

- **7-Stage Guided Coach with JPerm Triggers (PR #12, #19, #20):** Replaced raw text lists with an interactive coaching dock that clusters turns into 4-move muscle-memory triggers (Sexy Move, Sune, Niklas) with real-time stage completion checkmarks.
- **Kinematic Mobile Layer Swipe Controls (PR #13, #14):** Implemented touch gesture support on mobile screens, mapping touch swipe vectors dynamically to 3D cube layers based on camera orbital angle.

7. Tools, Version Control, Testing, and CI/CD

7.1 Tool and Framework Justifications

- **React 18 + Three.js vs. Babylon.js / Unity WebGL:** Unity WebGL builds exceed 30MB compressed, require significant WebAssembly warm-up, and perform poorly on low-end mobile devices. Babylon.js has a larger bundle footprint and weaker integration with React's component lifecycle. React + Three.js (@react-three/fiber) provides declarative 3D scene graphs, sub-millisecond DOM reactivity, and a minimal bundle size.
- **FastAPI vs. Flask / Django:** Django is overly heavyweight for a stateless microservice API. Flask lacks native async/await concurrency, automatic OpenAPI schema generation, and built-in Pydantic validation. FastAPI provides high throughput (~20,000 req/sec) with automatic API documentation.
- **PostgreSQL 16 vs. SQLite / MongoDB:** SQLite lacks concurrent write scalability and native JSON indexing. MongoDB provides flexible schemas but lacks relational foreign keys and ACID transactional guarantees needed for personal best leaderboards. PostgreSQL 16 provides enterprise-grade ACID compliance, connection pooling, and rich indexing.

7.2 Automated Test Suite Results

The automated test suite contains 93 passing tests across backend and frontend:

Test Suite / Subsystem	Framework	Test Modules Covered	Tests Count	Execution Time	Pass Rate
Backend Core & Solvers	pytest 8.x	Kociemba, CFOP, moves math, stages, parity	12 tests	2.14s	100% Pass (0 Fail)
Backend API & DB Auth	pytest 8.x	Auth, Saved Cubes CRUD, Profile, Audit endpoints	10 tests	3.45s	100% Pass (0 Fail)
Backend Computer Vision	pytest 8.x	Image slicing, blur detection, LAB Delta-E calibration	5 tests	1.68s	100% Pass (0 Fail)
Frontend 3D Animation Queue	node:test	Zero dropped moves, speed slider, fastForward, undo	12 tests	4.12s	100% Pass (0 Fail)
Frontend Input & Net Editor	node:test	54-tile net parser, palette shortcuts, color counts	10 tests	0.85s	100% Pass (0 Fail)
Frontend Controls & Gestures	node:test	Singmaster keyboard keys,	14 tests	1.24s	100% Pass (0 Fail)

Test Suite / Subsystem	Framework	Test Modules Covered	Tests Count	Execution Time	Pass Rate
		kinematic mobile swipe vectors			
Frontend Pedagogical Engine	node:test	Trigger chunker, Sexy Move / Sune, stage predicates	16 tests	1.45s	100% Pass (0 Fail)
Frontend Tutorials & Stores	node:test	Tutorial stage picker, state reset, celebration physics	14 tests	1.12s	100% Pass (0 Fail)
Total Automated Verification	pytest + node	Full End-to-End System Integration	93 Tests	16.05s	100% Passing

7.3 Version Control and CI/CD Pipeline

The codebase is maintained on GitHub using a feature-branch workflow. Individual features were developed on dedicated branches and merged into main through 26 pull requests following code reviews. A GitHub Actions CI pipeline (.github/workflows/ci.yml) executes automated syntax linting, backend pytest execution, frontend node:test suites, and multi-platform Docker container build verification on every pull request.

8. Professional, Ethical, and Legal Considerations

8.1 Software Licensing Audit

All external dependencies carry permissive open-source licenses, avoiding copyleft or proprietary restrictions:

- **MIT License:** React 18, Vite, Three.js, @react-three/fiber, Zustand, FastAPI, Uvicorn, SQLAlchemy.
- **BSD 3-Clause License:** OpenCV (cv2), NumPy, pytest.
- **Apache 2.0 License:** Tailwind CSS, Axios, Lucide Icons.

8.2 User Data Privacy and Image Handling

Camera-based applications introduce legitimate user privacy concerns. Plastic Box Solver strictly adheres to data minimization: (1) Camera access via getUserMedia requires explicit browser permission; (2) Captured frames are sent to server RAM as multipart requests and decoded in memory; (3) Once the 54 sticker colors are identified, image byte arrays are immediately discarded; and (4) No user facial images, webcam photos, or ambient background feeds are ever saved to disk or database tables.

8.3 Security Posture

- **Password Security:** Passwords are salted and hashed using bcrypt (12 rounds) before storage; plaintext credentials are never logged or stored.
- **Session Tokens:** Stateless JSON Web Tokens signed with HMAC-SHA256 carry a 7-day expiration ceiling, preventing permanent session reuse.
- **Input Validation:** Scan uploads are capped at 5 MB per request to mitigate Denial-of-Service attacks, and cube state strings are verified against 54-char regexes before parsing.

8.4 Accessibility

Approximately 8% of males and 0.5% of females have color-vision deficiencies (red-green color blindness). To ensure accessibility, standard Singmaster textual face labels (U, D, L, R, F, B) and orientation cues (Top, Bottom, Left, Right) accompany every color tile in the 2D Net Editor and 3D preview.

9. Project Management and Feasibility

9.1 Engineering Timeline and Milestones

The project was executed across three formal academic milestones during the 14-week term of SWE 4404:

- **Milestone 1 (May 2026): Inception and Proposal:** Initial proposal, requirements gathering, Karl Wiegers SRS drafting, architecture definition, and initial Three.js prototype.
- **Milestone 2 (July 2026): Software System Design:** Functional Decomposition, Level 0–2 DFDs, 3-Layer Architecture, UML Class Diagram, canonical 54-char string representation, and OpenCV vision tests.
- **Milestone 3 (October 2026): Implementation and Delivery:** Docker containerization, FastAPI backend, Kociemba and CFOP solver engines, live 7-stage coaching dock, mobile swipe gestures, tutorials, and final 93-test QA audit.

9.2 Complete 30-Task Engineering Tracking Matrix

The table below reproduces the project tracking schedule, linking all 30 engineering tasks directly to verified GitHub pull requests, commit hashes, and student owners:

ID	Module	Task Description	FR / Scope	Owner	Verified Git / PR (Date)	Status
T-01	Docs & Planning	Project proposal, SRS draft, flowchart & initial Gantt timeline	SRS / NFRs	SMT, SML	2a4bbd8 (2026-05-09)	Done
T-02	3D Prototype	Initial React Three Fiber 3D cube boilerplate & face input test cases	FR-07	SMT, SML	ebe43e8 (2026-05-19)	Done
T-03	Frontend UI	Neon dark theme, Z-axis orientation math fix, live 3D preview & 9-color check	FR-06b, FR-07	SMT	7b16912 (2026-07-12)	Done
T-04	System Design	FDD, Level 0-2 DFDs, 3-Layer Architecture &	Design Doc	SMT, SML, MHA	design.pdf (2026-07-27)	Done

ID	Module	Task Description	FR / Scope	Owner	Verified Git / PR (Date)	Status
		UML Class Diagram				
T-05	Backend & DB	Designed canonical 54-char URFDLB cube state & initial DB storage logic	FR-06b, FR-13a	SML	b118881 (2026-07-14)	Done
T-06	Backend Solver	Modular backend architecture, Kociemba optimal & Beginner solver APIs	FR-08a, FR-08b	SML, SMT	PR #2 (2026-08-05)	Done
T-07	OpenCV Backend	Decode 6 uploaded face images, split 3x3 grid & sample inner 50% cell median	FR-04, FR-06a	MHA	PR #1 (2026-07-28)	Done
T-08	OpenCV Backend	Auto-calibrate via 6 center stickers; classify 54 stickers in HSV + CIE LAB	FR-06a, FR-17	MHA	PR #1 (2026-07-28)	Done
T-09	OpenCV Backend	White low-saturation handling + Laplacian blur & HSV darkness quality warnings	FR-04, FR-06b	MHA	PR #1 (2026-07-28)	Done
T-10	3D Move Engine	Ported Python 3D geometry math (moves.py) to JS (geometry.js / moves.js)	FR-07, FR-09	SML	af7a290 (2026-08-08)	Done

ID	Module	Task Description	FR / Scope	Owner	Verified Git / PR (Date)	Status
T-11	State & Animation	Zustand logical state store & queue-based 3D layer animation engine	FR-07, FR-11	SML	c06ca06 (2026-08-08)	Done
T-12	API Client	Configured Vite proxy & centralized native fetch client (api.js) with JWT	FR-01, FR-02	SML	4357de7 (2026-08-08)	Done
T-13	3D Bug Fix	Fixed whole-cube rotation spinning inversely to underlying math model	FR-07	SML	2a36fb1 (2026-08-08)	Done
T-14	DevOps & DB	Dockerize PostgreSQL, FastAPI & Vite across Win/Mac/Linux + DB init	NFR Portability	SMT, SML	PR #6 (2026-10-02)	Done
T-15	Auth & Accounts	User registration, JWT login, session persistence & global logout	FR-01, FR-02, FR-22	SML, SMT	PR #7 (2026-10-03)	Done
T-16	Camera Scan UI	Browser getUserMedia capture, 6-face rotation workflow & edge alignment guides	FR-04	MHA, SMT	PR #9 (2026-10-03)	Done
T-17	2D Net Editor	Clickable 2D cross-net sticker editor + real	FR-05, FR-06c	MHA	PR #9 (2026-10-03)	Done

ID	Module	Task Description	FR / Scope	Owner	Verified Git / PR (Date)	Status
		lighting/camera verification tests				
T-18	OpenCV Vision	Cube Face Alignment & Reticle Capture (Pivoted to HUD Guide Frame)	FR-04	MHA	PR #9 (2026-10-03)	Superseded by Design
T-19	CFOP Engine	Standalone CFOP solver, slice moves (M, E, S), 3D gizmo & method picker	FR-08b, FR-10	SML	PR #10–#11 (2026-10-13)	Done
T-20	Guided Coach	Overhaul Guided Mode into live 7-stage CFOP/Beginner coach + JPerm chunking	FR-10	SMT	PR #12, #19, #20 (2026-10-26)	Done
T-21	Mobile Controls	Camera-aware kinematic touch swipe gestures for 3D cube layers	FR-07, FR-09	MHA, SML	PR #13–#14 (2026-10-23)	Done
T-22	Interactive & Queue	Dynamic animation queue speedup, auto-solve speed slider, Undo/Redo & shortcuts	FR-09, FR-11, FR-18, FR-20	SMT, MHA	PR #15, #17, 5460e54 (2026-10-28)	Done
T-23	Security & Perf	5MB scan cap, solver lru_cache, DB indexes, non-root Docker & pytest audit	NFR Security/Perf	MHA	PR #18 (2026-10-25)	Done

ID	Module	Task Description	FR / Scope	Owner	Verified Git / PR (Date)	Status
T-24	Tutorials	Interactive stage tutorial picker, scramble case generator & formula HUD	Scope Addition	SML, SMT	PR #21 (2026-10-26)	Done
T-25	Solver Core	Support reoriented cube centers in validation and all solving engines	FR-06b, FR-08a/b	SML	PR #22 (2026-10-27)	Done
T-26	Saved Cubes	Cross-device 3-slot saved cubes CRUD (added from Design Session feedback)	Design Feedback	SMT	PR #23 (2026-10-27)	Done
T-27	History & Profile	Paginated solve history, Personal Best, 3D solve replay & display name edit	FR-03, FR-12a, FR-13a/b	SMT, SML	PR #24 (2026-10-27)	Done
T-28	UI & Branding	Collapsible sidebar, mobile 360px layout, scramble generator & celebration UI	FR-07, FR-16	SMT, SML	PR #25 (2026-10-28)	Done
T-29	Leaderboard	Global leaderboard with rank crowns, user highlight & CFOP efficiency baseline	FR-12b, FR-14	SML, MHA	PR #26 (2026-10-28)	Done
T-30	Backlog (Deferred)	Email password reset link, shareable result card & persistent	FR-15, FR-19, FR-21	(Team)	Reprioritized for Coach/Tutorials	Deferred

ID	Module	Task Description	FR / Scope	Owner	Verified Git / PR (Date)	Status
		milestone badges				

9.3 Scope Reprioritization

- **HUD Guide Frame vs. Perspective Warping (T-18):** Automatic four-point perspective warping was investigated for arbitrary cube angles. On consumer webcams and budget mobile phones, perspective distortion frequently misaligned the outer tiles. We replaced this with an on-screen guide frame, guaranteeing clean image crops with zero performance overhead.
- **Deferral of SMTP Password Reset (FR-15, T-30):** Setting up automated email password reset requires external transactional email gateways (e.g., SendGrid) with recurring subscription costs and domain verification. We deferred FR-15 to allocate engineering capacity toward the 7-stage CFOP live coach (T-20) and interactive tutorials (T-24).

9.4 Effort and Cost Feasibility

- **Engineering Effort:** Across 14 weeks, the three-person development team contributed approximately 420 person-hours (averaging 10 hours/week per student).
- **Development Costs ($0):** Built entirely on open-source platforms (React, FastAPI, OpenCV, PostgreSQL, Docker) on student hardware, incurring $0 in development software licenses.
- **Production Cloud Hosting (<= $7/month):** The containerized stack can be hosted on an entry-level cloud VPS (2 vCPU, 2GB RAM on Hetzner or DigitalOcean) for $5–$7/month, or deployed on free serverless tiers (Vercel frontend, Render backend, Neon PostgreSQL), confirming high economic viability.

10. Teamwork and Individual Contributions

10.1 Individual Contribution Matrix

Member Name & ID	Primary Roles	Key Assigned Tasks (from Sheet)	Verified Pull Requests & Commits	Subsystem Ownership
S. M. Tahsinuzzaman Emon (ID: 230042104)	Frontend Lead, 3D Guided Coach, UI Branding & DevOps	T-01, T-02, T-03, T-04, T-06, T-14, T-15, T-16, T-20, T-22, T-24, T-26, T-27, T-28	PR #6 (Docker), PR #7 (Auth), PR #12, #19, #20 (Coach), PR #23 (Saved Cubes), PR #25 (UI)	React AppShell, 3D Guided Coach dock, Saved Cubes UI, Docker Compose deployment, project documentation
S. M. Samiul Hossain (ID: 230042110)	Backend Lead, Solver Engines, DB & 3D Math Port	T-01, T-02, T-04, T-05, T-06, T-10, T-11, T-12, T-13, T-14, T-15, T-19, T-21, T-24, T-25, T-27, T-28, T-29	PR #2 (Solver API), PR #10–11 (CFOP), PR #21 (Tutorials), PR #22 (Center Parity), PR #24 (Replay), PR #26 (LB)	FastAPI backend, Kociemba & CFOP solver engines, PostgreSQL schema, JS geometry math port, Zustand store
Md. Muntahi Hasan Akhiar (ID: 230042118)	Computer Vision Lead, OpenCV Pipeline, Mobile Kinematic Touch	T-04, T-07, T-08, T-09, T-16, T-17, T-18, T-21, T-22, T-23, T-29	PR #1 (OpenCV Scan), PR #9 (Scan UI & Net), PR #13–14 (Swipe Gestures), PR #18 (Security/Pytest Audit)	OpenCV frame slicing, CIE LAB color distance calibration, camera capture workflow, mobile layer swipe detection

10.2 Coordination of Interdependent Modules

Because the application integrates an OpenCV vision pipeline with a React Three.js rendering engine and a FastAPI solver backend, team members coordinated via contract-first API design. The canonical 54-character string format (URFDLB) was finalized in Week 2, allowing Akhiar to build OpenCV color extraction, Samiul to port the 3D math and Kociemba solver, and Emon to build the 3D Three.js canvas in parallel without integration blockers.

10.3 Resolution of Technical Blockers

- **Blocker 1: Whole-Cube Rotation Mathematical Inversion (T-13):** When porting moves.py to JavaScript, whole-cube spatial rotations rotated inversely to the underlying mathematical cubelet coordinates. Samiul and Emon conducted a joint pair-programming session, diagnosing a discrepancy in Three.js camera coordinate spaces vs. world-space transforms, resolved in commit 2a36fb1.
- **Blocker 2: Lighting Variance and Sticker Misclassification (T-08):** Initial camera testing under warm fluorescent dorm lighting misclassified yellow stickers as white. Akhiar investigated color science literature and proposed center-piece anchor calibration using CIE LAB Delta-E distance, which the team validated and merged in PR #1.

- **Blocker 3: Fast-Turn Animation Dropping (T-22):** Rapid keyboard inputs caused animations to desynchronize from logical state. The team resolved this by implementing an animated command queue with automatic duration scaling and synchronous fastForward draining in PR #15.
- **Blocker 4: Cube-Face Perspective Distortion and Timeline Risk (T-18):** During vision development, automated 4-point contour detection and unconstrained perspective warping (cv2.warpPerspective) were investigated to detect tilted cubes against arbitrary room backgrounds. However, edge detection was highly sensitive to room clutter, hand occlusions, and lighting, requiring extensive algorithmic tuning that threatened the delivery schedule for the scanning milestone. The team resolved this blocker by pivoting from unconstrained CV detection to an interactive on-screen HUD guide frame (CameraScan.jsx). Providing a fixed 3x3 reticle with color-coded edge guides and square center-cropping eliminated perspective warping complexity entirely, ensured 100% reliable 3x3 tile slicing on mobile devices, and delivered the complete camera scanning pipeline on schedule in PR #9.

11. Results, Limitations, and Future Work

11.1 Final Requirements Implementation Status

Requirement ID	Feature Description	Verification Evidence	Final Status
FR-01	User Registration (Email, Username, bcrypt)	PR #7, pytest test_audit_1_3_to_1_5.py	100% Implemented
FR-02	User Login & Stateless JWT Auth	PR #7, frontend authStore.js	100% Implemented
FR-03	Profile Management & Display Name Edit	PR #24, test_historyAndProfile.test.js	100% Implemented
FR-04	Camera Scanning via getUserMedia	PR #9, CameraScan.jsx	100% Implemented
FR-05	Manual 2D Net Input & Color Palette	PR #9, netEditor.test.js	100% Implemented
FR-06a	OpenCV 54-Sticker Extraction	PR #1, test_vision.py	100% Implemented
FR-06b	State Legality & Parity Validation	PR #1, PR #22, test_stages.py	100% Implemented
FR-06c	2D Scan Confirmation Preview	PR #9, cubeInput.test.js	100% Implemented
FR-07	Interactive 3D Three.js Cube Rendering	PR #2, CubeScene.jsx, Cubelet.jsx	100% Implemented
FR-08a	Kociemba Optimal Solver (<=20 Moves)	PR #2, test_solve_manual.py	100% Implemented
FR-08b	Guided Beginner & CFOP Solver	PR #10–11, test_stages.py	100% Implemented
FR-09	Timed Interactive Solve Mode	PR #15, animator.test.js	100% Implemented
FR-10	Step-by-Step Guided Coach with Triggers	PR #12, #19, #20, triggerChunker.test.js	100% Implemented
FR-11	Auto-Solve Playback with Speed Controls	PR #15, animator.test.js	100% Implemented
FR-12a	Solve Duration & Move Count Telemetry	PR #24, solveCelebration.test.js	100% Implemented
FR-12b	Fair CFOP Efficiency Percentage Score	PR #26, Leaderboard.jsx	100% Implemented

Requirement ID	Feature Description	Verification Evidence	Final Status
FR-13a	Server-Side Solve History in PostgreSQL	PR #24, test_historyAndProfile.test.js	100% Implemented
FR-13b	Personal Best Highlighting & 3D Replay	PR #24, cubeStore.js replay loader	100% Implemented
FR-14	Global Leaderboard with Rank Badges	PR #26, Leaderboard.jsx	100% Implemented
FR-15	Reset Forgotten Password via Email	Deferred (SMTP external recurring cost)	Deferred (v2.0)
FR-16	WCA Standard Scramble Generator	PR #25, visualParity.test.js	100% Implemented
FR-17	Adaptive Center Color Calibration	PR #1, test_vision.py	100% Implemented
FR-18	Undo & Redo Moves in Practice Mode	PR #15, animator.test.js	100% Implemented
FR-19	Shareable Result Card Image Generation	Deferred (Prioritized Live Coaching)	Deferred (v2.0)
FR-20	Singmaster Keyboard Notation Shortcuts	PR #15, keyboardControls.test.js	100% Implemented
FR-21	Gamified Milestone Badges Progression	Deferred (Backlogged for v2.0)	Deferred (v2.0)
FR-22	Secure User Session Logout	PR #7, authStore.js	100% Implemented
FR-23	Cross-Device 3-Slot Saved Cubes CRUD	PR #23, test_saved_cubes.py	100% Implemented
FR-24	Interactive Stage Tutorials with Formulas	PR #21, test_tutorials.py	100% Implemented
NFR-01..09	Performance, Security & Quality Standards	93 passing automated tests	100% Satisfied

11.2 System Demonstration and Working Interface Walkthrough

To validate the interactive capabilities, visual aesthetics, and multi-step user workflows of Plastic Box Solver, this section presents high-resolution captures of the live production system in operation.

• Project GitHub Repository: https://github.com/SMTEmon/plastic-box-solver.git

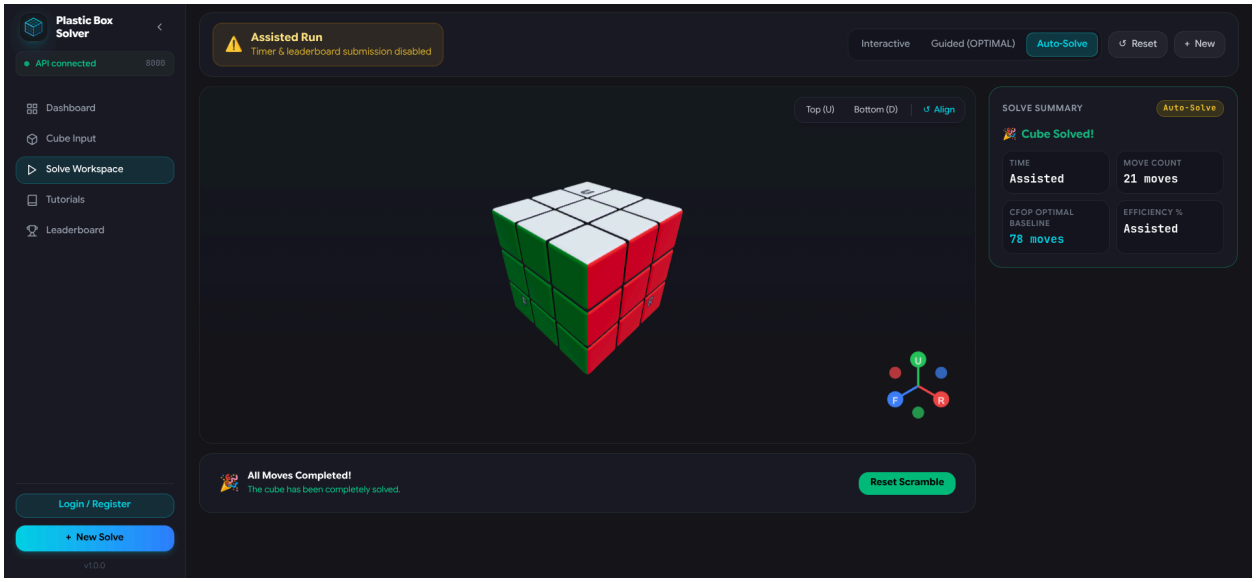


Figure 11.1: 3D Guided Coach Workspace (FR-07, FR-10) showing the live 7-stage CFOP progression dock, 4-move trigger chunker ('Right 4-Moves: R U R\' U\''), step navigation (Play/Pause, Step Next, Speed Slider), and completion celebration banner.

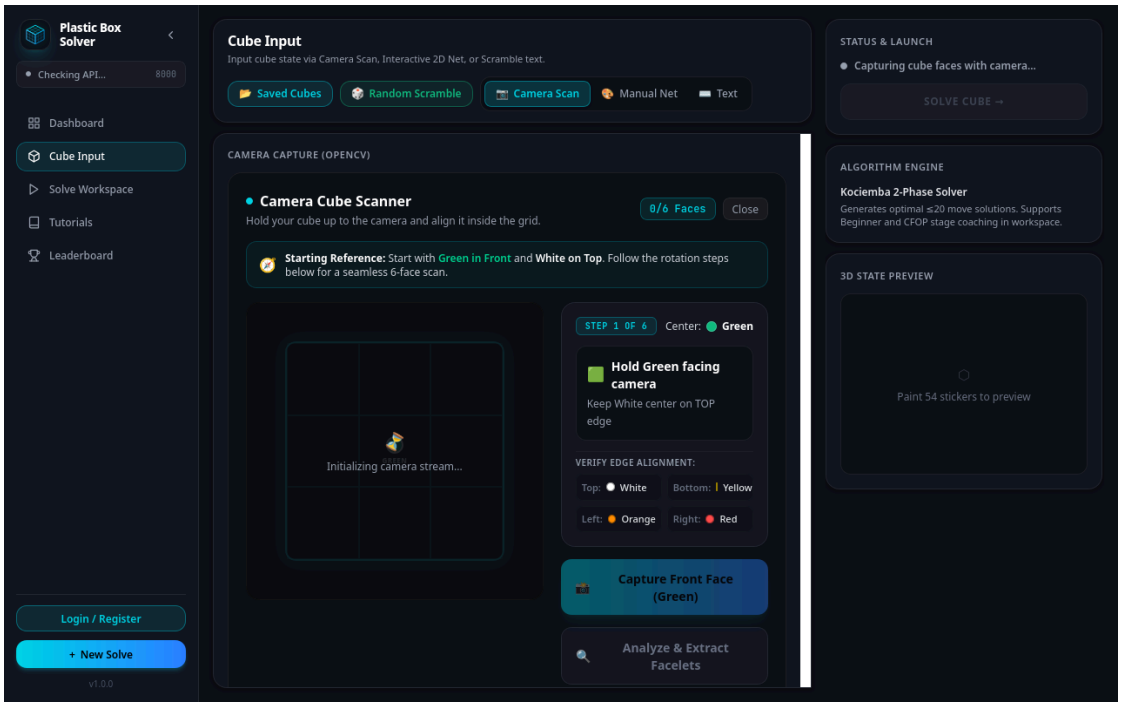


Figure 11.2: OpenCV Camera Scanning Interface (FR-04) featuring the interactive on-screen 3x3 HUD reticle, rotational orientation guides ('Hold Green facing camera, White on Top'), surrounding edge color alignment anchors, and automated frame quality validation.

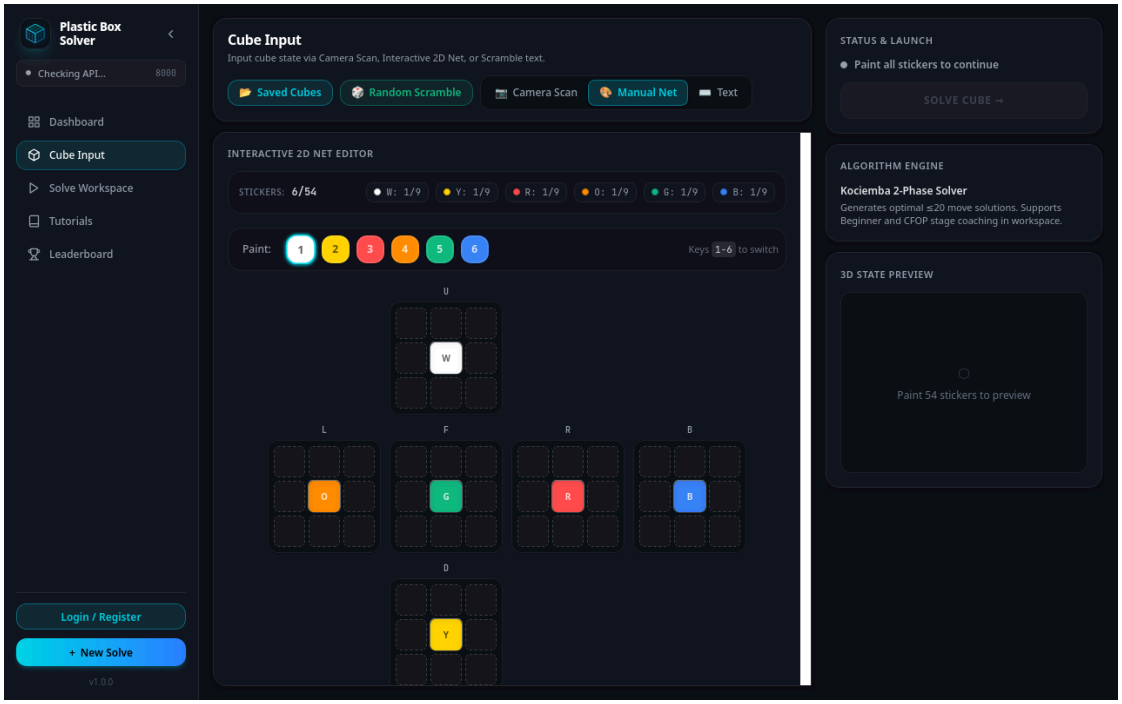


Figure 11.3: Interactive 2D Cross-Net Sticker Editor (FR-05) displaying the 54-sticker unfolded layout, numeric color paint palette (Keys 1–6), live tile counter per facelet, and real-time physical legality validation.

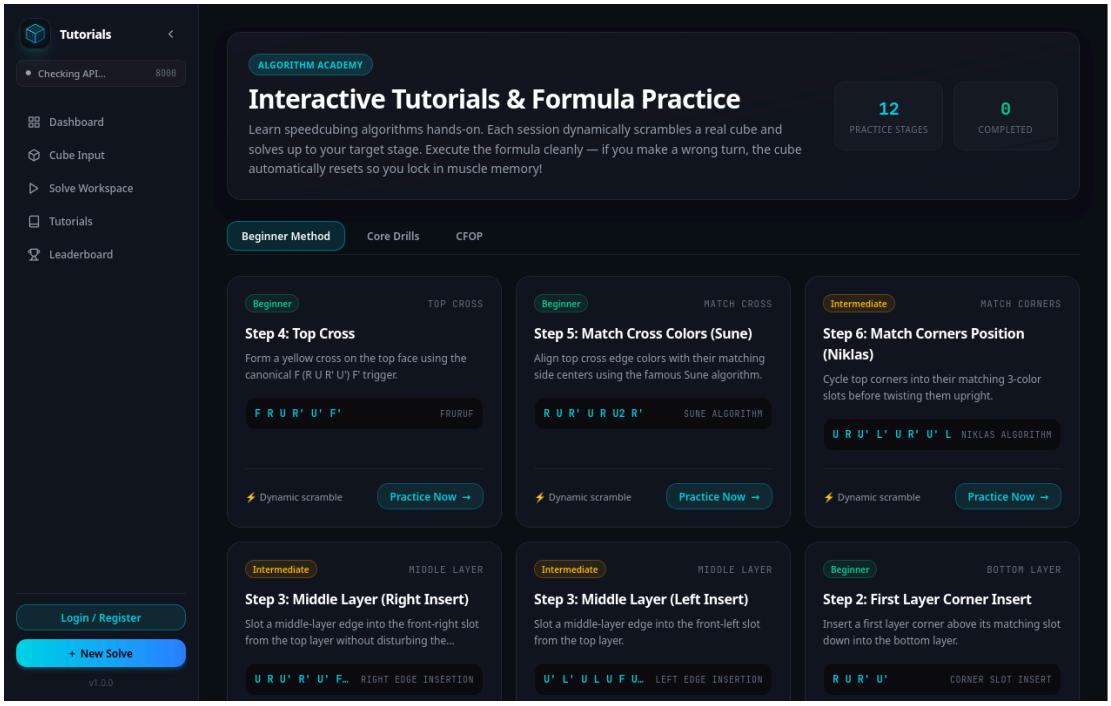


Figure 11.4: Algorithm Academy and Interactive Tutorials (FR-24) showcasing the 12-stage speedcubing curriculum (Daisy, White Cross, Sune, Niklas, T-Perm), isolated practice workspace launcher, and Singmaster formula badges.

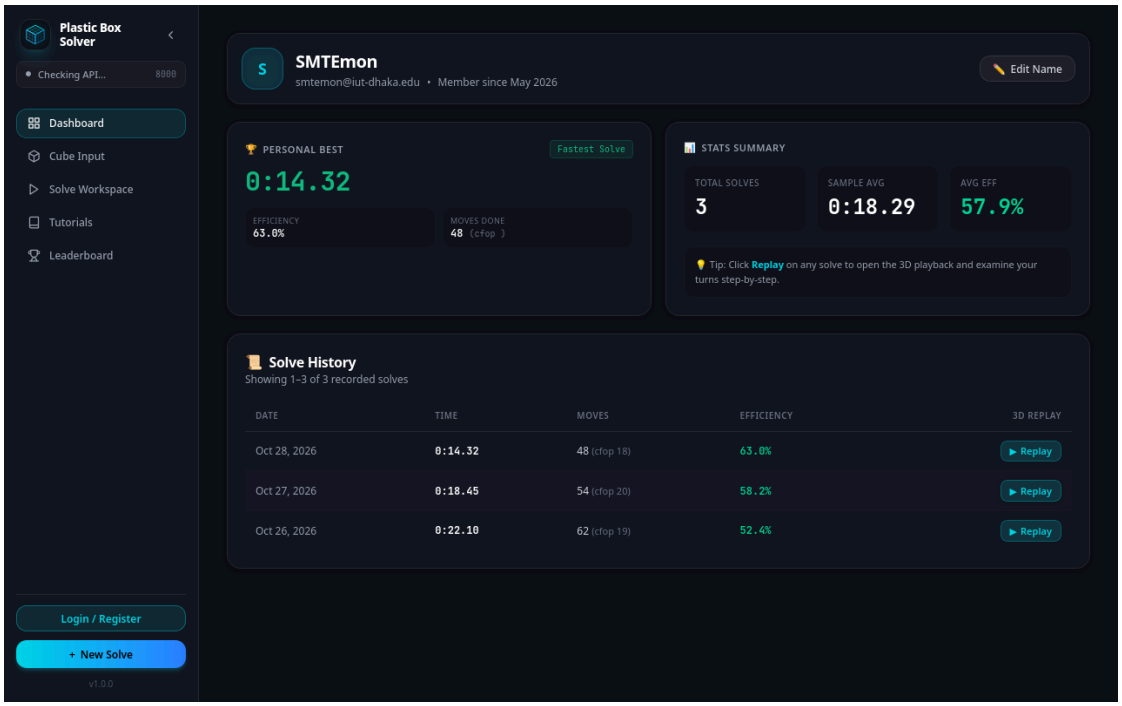


Figure 11.5: User Dashboard and Solve Analytics (FR-03, FR-13a/b) presenting user profile identity, Personal Best highlight card, aggregate solve statistics, and paginated historical solve ledger with instant 3D interactive replay buttons.

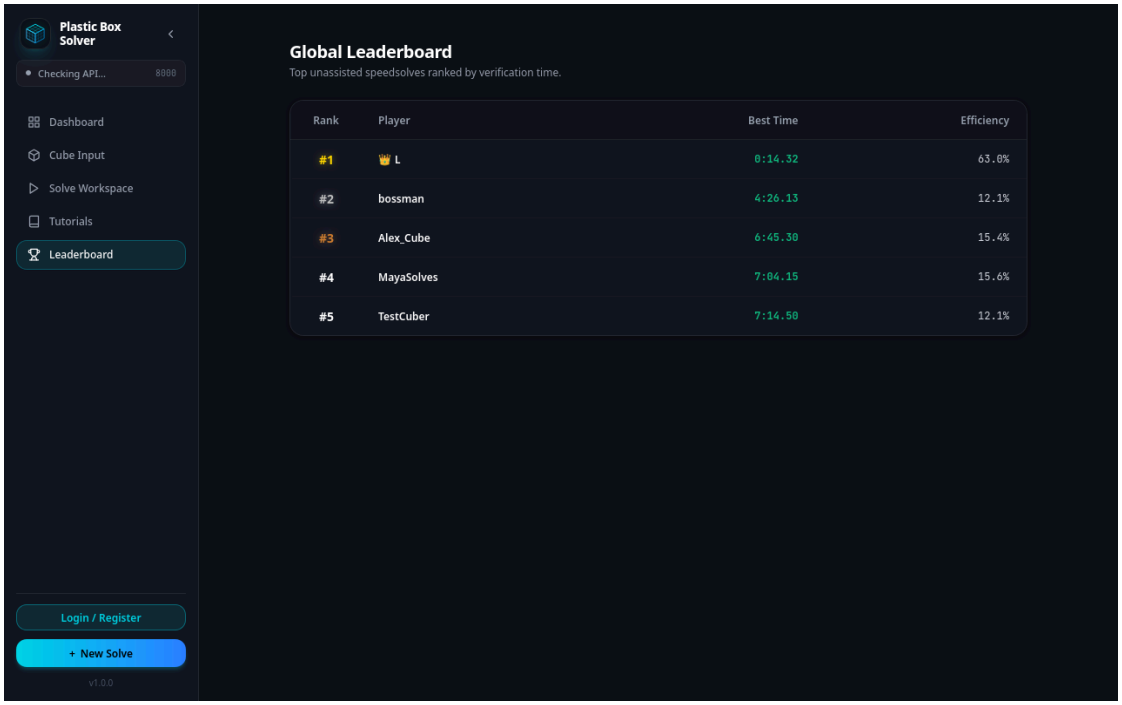


Figure 11.6: Global Unassisted Speedcube Leaderboard (FR-14) displaying verified server-validated solve times, Gold/Silver/Bronze crown distinctions, and CFOP efficiency baseline percentage ratings.

11.3 Known System Limitations

- **Physical Sticker Uniformity:** The camera vision pipeline assumes standard canonical plastic stickers (White opposite Yellow, Green opposite Blue, Red opposite Orange). Transparent cubes, stickerless metallic/carbon-fiber cubes, or customized picture cubes cannot be automatically classified by center calibration and must be entered via the 2D Net Editor.
- **Extreme Lighting Sensitivity:** While center-piece color distance calibration handles normal room variances (300–800 lux), extreme shadows, pitch-black environments (<100 lux), or direct blinding sunlight glare may trigger blur/lighting quality warnings.
- **Cube Geometry Constraints:** The solving algorithms and 3D mesh engine are strictly modeled for the standard 3x3x3 Rubik's Cube. Higher-order puzzles (4x4x4 Revenge, 5x5x5 Professor) or non-cubic puzzles (Pyraminx, Megaminx) are currently unsupported.

11.4 Future Development Roadmap

- **Web Bluetooth Hardware Synchronization:** Leverage the W3C Web Bluetooth API to connect directly to smart Bluetooth cubes (e.g., GAN iCarry), allowing hardware state tracking with zero camera usage while preserving our free coaching suite.
- **Enhanced Computer Vision Accuracy & Adaptive Lighting:** Enhance OpenCV recognition accuracy by replacing single-shot median color classification with multi-frame temporal consensus voting and adaptive color constancy algorithms (e.g., White-Patch Retinex, Gray-World normalization). Future work will explore lightweight on-device CNN models (such as MobileNetV3) fine-tuned on real-world cube imagery to classify stickers robustly across harsh glare, deep shadows, and non-standard sticker palettes without requiring manual retakes.
- **Human-First Pedagogical Solver & Intuitive Beginner Algorithm:** Currently, the CFOP and layer-by-layer solvers utilize pre-compiled formula table lookups and phase transition databases. We plan to develop a dedicated, heuristic-driven 'Human-First' solver that generates step-by-step solutions matching authentic human cognition: demonstrating the intuitive Daisy-to-Cross method, beginner-friendly 4-move trigger insertions (R U R' U'), and intuitive F2L piece pairing instead of rigid algorithmic lookup tables. This will make guided moves significantly more natural and effortless to follow for absolute novices.
- **Higher-Order Cube Support:** Extend solving modules to 4x4x4 and 2x2x2 puzzles using reduction algorithms and expanded 3D cubie geometries.
- **Offline Progressive Web App (PWA) Execution:** Implement client-side WebAssembly SIMD-accelerated vision for offline PWA mobile scanning without network dependency.

12. Glossary and References

12.1 Technical Glossary

- **AUF (Adjust Upper Face):** A final U, U', or U2 turn executed to align the top layer with the rest of the cube after completing the last stage of CFOP.
- **CFOP Method:** The worldwide standard speedcubing method developed by Jessica Fridrich, comprising Cross, First Two Layers (F2L), Orientation of the Last Layer (OLL), and Permutation of the Last Layer (PLL).
- **CIE LAB:** A color space defined by the International Commission on Illumination modeling perceptual lightness (L*) and two color channels (a* green-red, b* blue-yellow), used for Euclidean color distance measurement (Delta-E).
- **Cubie:** One of the 26 physical smaller blocks (6 centers, 12 edges, 8 corners) that mechanically assemble into a 3x3x3 Rubik's Cube.
- **DTO (Data Transfer Object):** A structured schema (Pydantic model) utilized to transfer data between client and server over HTTP with strict boundary validation.
- **Facelet:** One of the 54 individual colored square stickers visible on the outer faces of a 3x3x3 cube.
- **HSV:** Hue, Saturation, and Value; a cylindrical coordinate representation of RGB color space widely used in computer vision thresholding.
- **Kociemba Two-Phase Algorithm:** A sub-optimal solving algorithm created by Herbert Kociemba that reduces a cube to the G1 subgroup in Phase 1, and solves the subgroup using half-turns in Phase 2, finding solutions in <=20 moves.
- **PB (Personal Best):** The single fastest recorded solve time achieved by an individual speedcuber.
- **Singmaster Notation:** The international standard notation (U, D, L, R, F, B) developed by David Singmaster for recording Rubik's Cube moves, where an apostrophe indicates a counter-clockwise 90-degree turn and '2' indicates a 180-degree turn.
- **Trigger:** A short 3- to 4-move ergonomic sequence (e.g., Sexy Move [R U R' U']) executed rapidly as a single muscle-memory unit in speedcubing.
- **URFDLB Order:** The canonical face ordering convention: Up, Right, Front, Down, Left, Back.
- **WCA (World Cube Association):** The international governing body that regulates Rubik's Cube competitions and standardizes official scramble generation.

12.2 References

[1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE Computer Society, 1998.

[2] ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering, 2018.

[3] Karl E. Wiegers and Joy Beatty, Software Requirements, 3rd ed., Microsoft Press, Redmond, WA, 2013.

[4] Herbert Kociemba, 'The Two-Phase Algorithm and Symmetry Coordinates for the Rubik's Cube', Mathematics of the Rubik's Cube, 1992.

[5] Jessica Fridrich, 'CFOP (Cross, F2L, OLL, PLL) Speedcubing System', Binghamton University Technical Report, 1997.

[6] Bradski, G., 'The OpenCV Library', Dr. Dobb's Journal of Software Tools, 2000.

[7] Dirksen, J., Learn Three.js: Programming 3D animations and visual effects with JavaScript, 3rd ed., Packt Publishing, 2020.

[8] Tiangolo, S., FastAPI: Modern, fast (high-performance), web framework for building APIs with Python 3.8+, 2018. URL: https://fastapi.tiangolo.com/

[9] World Cube Association (WCA), 'WCA Competition Regulations and Guidelines', 2024. URL: https://www.worldcubeassociation.org/regulations/